

Epic 2: Core Functionalities

1. Core Feature Capabilities

EduGenie provides five specialized educational functionalities designed to help students and instructors analyze, learn, and test knowledge effectively. Each capability uses targeted context instructions to deliver structured, meaningful output.

Concept Explanation: Provides clear, concise, and direct explanations of technical terms or theories. It runs locally to ensure fast, reliable access without external network dependencies.

General Q&A; Engine: Accepts open-ended questions and generates clear, detailed answers enriched with practical, real-world examples.

Summarization Engine: Condenses long, complex multi-paragraph texts into a structured format, providing a concise summary introduction followed by punchy bullet points.

Quiz Generation System: Automatically builds a custom three-question multiple-choice assessment based on input reading material, complete with a verified answer key and rationales.

Learning Path Architect: Generates chronological, multi-phase study roadmaps that guide users step-by-step from beginner basics to advanced mastery.

2. Technical Code Implementations and SDK Interfacing

The cloud functionalities use a standardized design pattern centered around the google-genai client library. The client is instantiated once at server startup using verified credentials and passed down to individual sub-modules to preserve memory resources.

The system uses structured prompting wrappers to ensure consistent output formatting across different tasks. For instance, `quiz_module.py` wraps user inputs with explicit rules that force options into an (A, B, C, D) layout and mandate an 'Answer Key' section at the end. Similarly,

learning_path.py injects requirements for clear chronological phases, time estimates, and markdown headers.

The local module (explanation_module.py) takes a different approach, using an internal singleton pattern to load AutoTokenizer and AutoModelForSeq2SeqLM into memory only once. This script processes input strings through a local tensor pipeline, limits generation length to 256 new tokens, and decodes the result directly on the host CPU.